

PROYECTO FINAL - NEST : WINTER EDITION

Tyler Zarzuela Amor

1. Temática y objetivo de juego

Nest: Winter Edition está basado en el juego de estrategia online *Nest*, donde se debe llevar al personaje principal, un pajarito, de vuelta a su nido. A diferencia del original, se trata de un juego en 3D ambientado en un paisaje invernal con distintos fenómenos meteorológicos por nivel, algunos más tranquilos con nieve cayendo suavemente y otros más intensos con tormentas gélidas. Debido a que el pajarito es muy pequeño, aún no sabe cómo volar correctamente, así que no deberá tocar el suelo o se congelará. Tampoco podrá chocar con objetos dañinos o morirá.

Esta versión del juego mantiene la mecánica básica de cambio de forma, permitiendo tanto al jugador como a los enemigos alternar entre un cubo y una esfera. Esto permitirá al jugador planificar mejor sus movimientos para poder completar los niveles con éxito. También cuenta con una mecánica nueva: el viento, el cual podrá ser activado a través de ventiladores.

Comenzando por una pantalla principal, el juego proporciona al jugador la información necesaria sobre el objetivo y los controles utilizados.

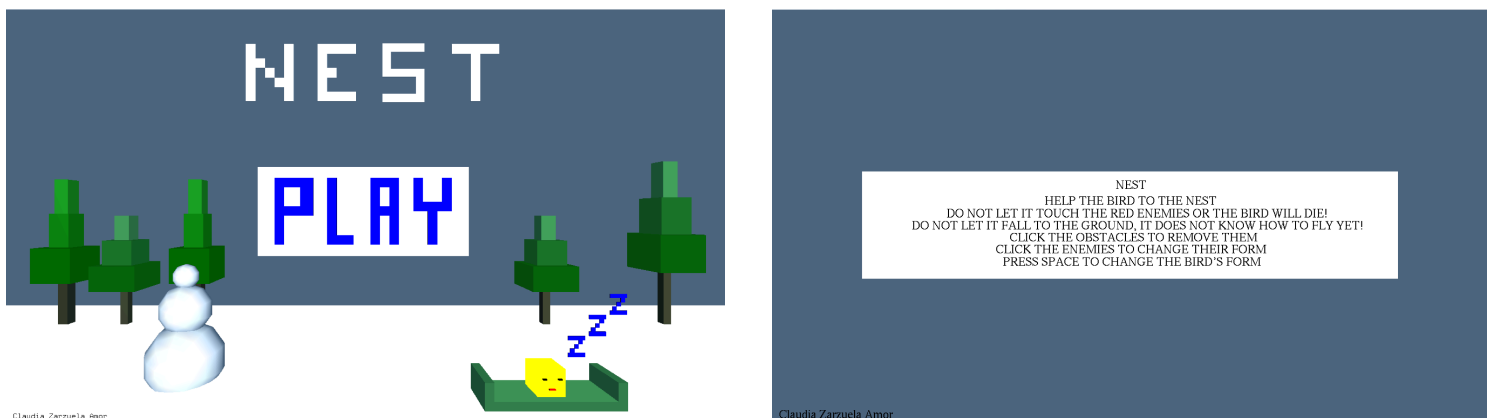
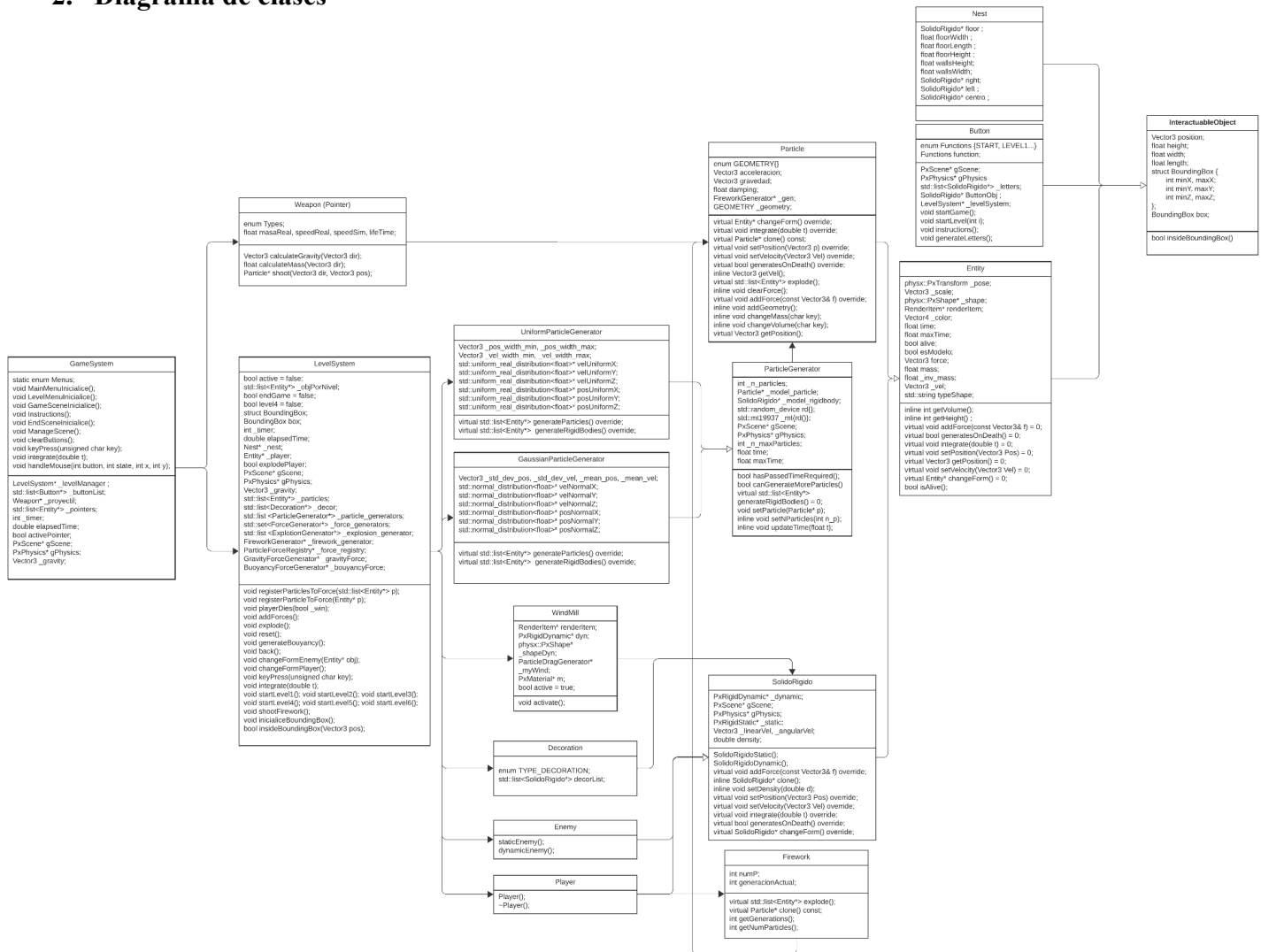


Imagen 1. Pantalla de inicio e introductoria con objetivo y controles



Imagen 2. Pantalla de niveles

2. Diagrama de clases



Para más información y mejor entendimiento de la lógica interna de juego, el código contiene breves explicaciones en cada clase nueva de su funcionamiento. Las clases más importantes son el **GameSystem**, clase principal encargada de la gestión de menús así como de la creación de los "*pointers*", y el **LevelSystem**, clase encargada de la creación y del desarrollo de mecánicas y funcionalidad de los niveles.

También se puede encontrar en GitHub la versión completa de UML, con todas las clases.

3. Ecuaciones físicas usadas

Para toda la física relacionada con las mecánicas básicas del juego como las colisiones o la gravedad se usa el motor físico propio de Nvidia, excepto para simular el viento de los ventiladores, para el cual se usa el Drag Force Generator en los enemigos, sólidos rígidos dinámicos con distinta masa, densidad y forma, del nivel.

Sin embargo, para poder poner en contexto al jugador y enfatizar la historia y objetivo de juego se usan distintos tipos de fuerzas con el objetivo de simular distintos ambientes invernales y momentos del día. Se emplean las siguientes fuerzas mostradas:

Drag Force: usado con los generadores de entidades, varían dependiendo de la intensidad de los fenómenos meteorológicos de cada nivel. También usado para generar el viento de los ventiladores.

Vector3 v = velocidad de la partícula;

Vector3 $\text{dragF} = _k1 * (_wind - v) + _k2 * (_wind - v).magnitude() * (_wind - v);$

Valores usados:

_k1	_k2	_wind
1	0	<i>Ventiladores</i> = {varían en un rango desde -10 hasta 10 en la x , valor 0 en la y , valor 0 en la z } <i>Metereología</i> = {varían en un rango desde -60 hasta 150 en la x , varían en un rango desde -100 hasta 0 en la y , valor 0 en la z }

WhirlWind Force: usado con generadores de partículas para generar tormentas gélidas.

Vector3 pos = posición de la partícula;

Vector3 posOrigen = posición origen del torbellino;

$_wind = K * \text{Vector3}(-(\text{pos.z} - \text{posOrigen.z}) * 7, ((\text{pos.y} - \text{posOrigen.y})), (\text{pos.x} - \text{posOrigen.x}));$

Valores usados:

K	2
----------	---

Gravity Force: gravedad añadida a las nuevas partículas creadas por los generadores. Los sólidos rígidos usan la fuerza de la gravedad proporcionada por *physx*.

Vector3 $f = \text{gravity} * \text{masa de la partícula};$

Valores usados:

gravity	9,8
----------------	-----

Buoyancy Force: usado para generar un gran lago azul con cubitos de hielo de distinta masa, densidad y forma flotando sobre él a lo lejos.

float h = posición en y de la partícula; **float** h0 = posición en y la la partícula líquido;

float immersed = 0.0; **float** h = altura de la partícula

```
if (h - h0 > particle->getHeight() * 0.5) immersed = 0.0;
else if (h0 - h > particle->getHeight() * 0.5) immersed = 1.0;
else immersed = (h0 - h) / particle->getHeight() + 0.5;
Vector 3 f = {0, _liquid_density *volumen de partícula * immersed * 9.8, 0}
```

Valores usados:

_liquid_density	1000
-----------------	------

4. Efectos incorporados

Además de los generadores uniformes y gaussianos empleados con las distintas fuerzas para mejorar la estética general del juego también se han generado distintos objetos como decoración (ej: árboles, muñecos de nieve, etc.) y fondos para distinguir los distintos momentos del día de cada nivel.

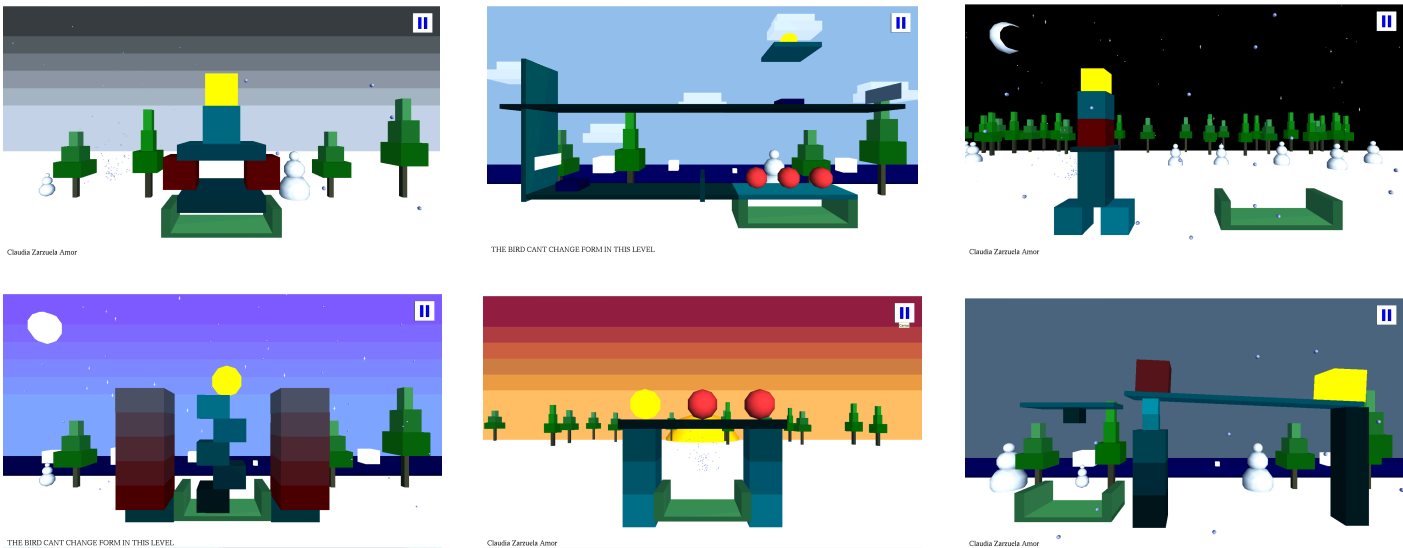


Imagen 3. Diferentes fondos y decoración por cada nivel.

El jugador deberá eliminar los objetos en escena estratégicamente y así guiar al pajarito a su nido. Para poder eliminarlos deberá apuntar y hacer click en ellos. Al presionar el botón izquierdo del ratón se disparará una bala desde la posición de pantalla donde esté situado el ratón, con velocidad $(0,0,-1000)$ para que actúe como puntero. Todos los objetos en escena heredan de una clase común *InteractableObjects*, la cual calcula si dado un objeto cualquiera este colisiona con él teniendo en cuenta la posición del objeto con respecto su bounding box.

Las colisiones de los punteros pueden ser:

- **Puntero - Botón** : se llama a la función asociada de ese botón. Usado para la gestión de niveles.
- **Puntero - Enemigo**: cambia de forma al enemigo, excepto al suelo, el cuál también se considera dañino
- **Puntero - Objeto Normal**: lo elimina de escena

Durante el juego, el pajarito también podrá cambiar de forma al igual que sus enemigos. Si este choca con un enemigo, este se eliminará de la escena y saldrá en su lugar unos fireworks rojos para indicar su derrota. Sin embargo, en caso de victoria, si consigue entrar en su nido, saldrá en su lugar unos fireworks verdes.

5. Manual de usuario

Durante el juego, el ratón será lo más importante. Al hacer clic izquierdo, disparará una bala en la posición exacta del ratón en pantalla. Esta bala actuará como puntero para pulsar botones, eliminar objetos en pantalla y cambiar de forma a los enemigos rojos. Para cambiar de forma al pajarito amarillo, personaje principal, se debe pulsar la tecla de *espacio*.

6. Efectos o experimentos extras

Ya que las escenas generadas por nvidia son elásticas, todos mis sólidos rígidos tienen asociados materiales inelásticos, detección de colisiones con mayor distancia, *filtros de datos* para mejorar las colisiones entre ellos y un tensor de inercia con la componente en x a 0 para simular un mundo en 2d y evitar que giren en ese eje. Así mismo, he añadido *flags* al main que me permiten controlar mejor las colisiones y he disminuido el tiempo entre frames para que los cálculos sean mayores, más precisos, y no pierda información.